A Brief Primer on Transform Coding

Shuvraneel Mitra

May 15, 2026

Abstract

We look at how transform coding works, its basic principles and a short look at the K-L transform: an "optimal" transform in terms of the least-square error sense.

1 Introduction

Lossy image compression techniques work in the spatial domain, since we are predicting the pixel values and the prediction errors (which are teh values being encoded) also correspond to the pixel space. It is seen that the compression ratios of Differential Pulse Code Modulation (DPCM) encoded images are not always very high. This is primarily due to the fact that in presence of sharp changes in intensity values, which are always expected in any natural image, prediction suffers and encoding large prediction errors in those regions lead to high consumption of bits.

In transform-domain approaches, the pixel intensities are first mapped into a set of **linear, reversible** transform coefficients, which are subsequently quantized and encoded.

2 Transform Coding

The basic principle of transform coding is to map the pixel values into a set of linear transform coefficients, which are subsequently quantized and encoded. By applying an inverse transformation on the decoded transform coefficients, it is possible to reconstruct the image with some loss. It must be noted that *the loss is **not** due to the process of transformation and inverse transformation, but due to quantization* alone.

Since the details of an image and hence its spatial frequency content vary from one local region to the other, it is better to apply the transformation on local areas of the image, rather than applying a global transformation on the entire image.

For transform coding, the first and foremost step is to subdivide the image into non-overlapping blocks of fixed size. Without loss of generality, we can consider a square image of size $N \times N$ pixels and divide it into n^2 blocks, each of size $\frac{N}{n} \times \frac{N}{n}$, where $n << N$ and is a factor of N.

2.1 What kinds of transforms are allowable?

A transformation must necessarily fulfill the following properties –

1. The coefficients in the transformed space should be de-correlated.

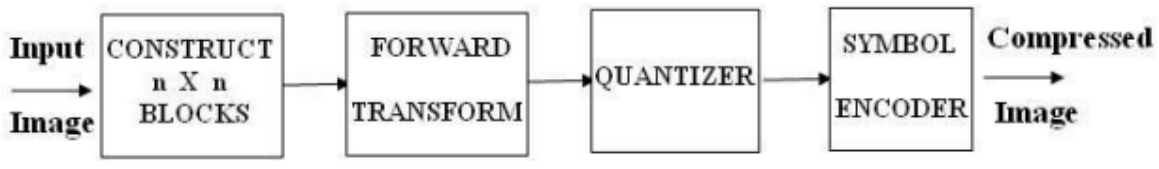

Figure 1: Block Diagram of Transform Coding System Encoder

Figure 2: Block Diagram of Transform Coding System Decoder

2. Only a limited number of transform coefficients should carry most of the signal energy (in other words, the transformation should possess energy compaction capabilities) and most of the coefficients should carry insignificant energy. Then the quantization process can coarsely quantize those "extra" coefficients to achieve compression, without much perceptible degradation.

2.2 Generalized forward and inverse transforms

Consider an image block of size $n \times n$ whose pixel intensities are represented as $s[n_1, n_2]$, with $n_1, n_2 = 0, 1, \ldots, n-1$. Then the general expression for the transform coefficients is given as

$$S[k_1, k_2] = \sum_{n_1=0}^{n-1} \sum_{n_2=0}^{n-1} s[n_1, n_2] g(n_1, n_2, k_1, k_2) \tag{1}$$

for $k_1, k_2 = 0, 1, ..., n-1$, where $g(n_1, n_2, k_1, k_2)$ is the transformation kernel.

Given the transformation coefficients, the original image can be regained as

$$s[n_1, n_2] = \sum_{k_1=0}^{n-1} \sum_{k_2=0}^{n-1} S[k_1, k_2] h(n_1, n_2, k_1, k_2) \tag{2}$$

for $n_1, n_2 = 0, 1, ..., n-1$, where $h(n_1, n_2, k_1, k_2)$ is the inverse transform kernel.

2.3 Separable and Symmetric Transforms

A transformation kernel is said to be separable if it can be expressed as a product of two kernels along the row and the column:

$$g(n_1, n_2, k_1, k_2) = g_1(n_1, k_1) g_2(n_2, k_2)$$

Separable transforms are easier to implement in hardware, since the transformation can first be applied along the rows and then along the columns.

A separable transform is symmetric, if the kernels along the row and the column have the identical function, i.e. $g_1 = g_2$.

2.4 Basis Images

From 2 we have n^2 such equations, one for each pair (n_1, n_2). These equations can be combined in matrix form to read as:

$$\mathbf{s} = \sum_{k_1=0}^{n-1} \sum_{k_2=0}^{n-1} S[k_1, k_2] \mathbf{H}_{k_1, k_2}$$

where

$$\mathbf{H}_{k_1,k_2} = \begin{bmatrix} h(0,0,k_1,k_2) & h(0,1,k_1,k_2) & \cdots & h(0,n-1,k_1,k_2) \\ h(1,0,k_1,k_2 & \cdots & & \\ \vdots & \ddots & \cdots & \vdots \\ h(n-1,0,k_1,k_2) & h(n-1,1,k_1,k_2) & \cdots & h(n-1,n-1,k_1,k_2) \end{bmatrix}$$

The image block $\mathbf{s}$ can therefore be realized by a weighted summation of n^2 images, each of size $n \times n$, where the weights are provided by the transform coefficients $S(k_1, k_2)$. The matrix $\mathbf{H}_{k_1,k_2}$ is known as a basis image corresponding to (k_1, k_2) . There are n^2 such basis images.

3 The Covariance Matrix

A block may be represented by a n^2 - dimensional random variable vector $\mathbf{x}$, whose elements are composed by the lexicographic ordering of pixel intensity values. We define a vector $\mathbf{b}$, such that $\mathbf{b} = \mathbf{x} - \mathbb{E}[\mathbf{x}]$. We approximate $\mathbb{E}[\mathbf{x}] = \mu$, where $\mu = \frac{1}{N_B}\sum_i \mathbf{x}_i$ where N_B is the number of $n \times n$ blocks present.

The covariance matrix computed over blocks of size $n \times n$ is defined by

$$\mathbf{R}_b = \mathbb{E}[\mathbf{b}\mathbf{b}^T] = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T]$$

where, as before, the expectation is approximated by averaging over all the blocks. Since $\mathbf{b}$ is an n^2 dimensional vector, its outer product realizes an $n^2 \times n^2$ - dimensional matrix, which is the size of $\mathbf{R}_b$. The matrix is real and symmetric and it is possible to find a set of n^2 orthonormal eigenvectors. (**Why?**)

Let $\mathbf{e}_i$ and λ_i, $i = 1, 2, \ldots, n^2$ be the eigenvectors and corresponding eigenvalues such that the eigenvalues are arranged in non-increasing order such that $\lambda_j \geq \lambda_{j+1}$. Then, by the definition of eigenvectors and eigenvalues, we can write

$$\mathbf{R}_b\mathbf{e}_j = \lambda_j\mathbf{e}_j, \qquad j = 1, 2, ..., n^2$$

Since $\mathbf{e}_j^T\mathbf{e}_j = 1$ we can pre-multiply $\mathbf{e}_j^T$ to get

$$\mathbf{e}_j^T\mathbf{R}_b\mathbf{e}_j = \lambda_j \tag{3}$$

Now compose a matrix of Γ dimension $n^2 \times n^2$ such that its rows are formed from the eigenvectors of $\mathbf{R}_b$, ordered such that the first row corresponds to the eigenvector corresponding to the largest eigenvalue and the corresponding values of the eigenvalues decreases thereafter. Then, we can write 3 in matrix form as:

$$\Gamma\mathbf{R}_b\Gamma^T = \Lambda \tag{4}$$

, where Λ is the diagonal matrix of ordered eigenvalues (why this "ordering" is necessary will be revealed soon).

Noting that by the orthonormal properties of Γ, we get $\Gamma^T = \Gamma^{-1}$ and so we obtain

$$\mathbf{R}_b = \Gamma^T\Lambda\Gamma$$

4 Using the covariance matrix: The Karhunen-Loeve (K-L) Transform

We intend to use the matrix Γ to map the n^2-dimensional vector $\mathbf{b}$ to an n^2-dimensional vector $\mathbf{y}$, defined by $\mathbf{y} = \Gamma\mathbf{b}$. This is the **K-L Transform**.

The covariance matrix $\mathbf{R}_y$ of the $\mathbf{y}$ vectors is given as

$$\mathbf{R}_y = \mathbb{E}[yy^T] = \mathbb{E}[(\Gamma\mathbf{b})(\Gamma\mathbf{b})^T] = \Gamma\mathbb{E}[\mathbf{b}\mathbf{b}^T]\Gamma^T = \Gamma\mathbf{R}_b\Gamma^T = \Lambda \qquad \text{(by (4))}$$

The covariance matrix has the same eigenvectors and eigenvalues as that of $\mathbf{R}_b$, but its off-diagonal elements are zero, which signifies that **the elements of the transform-domain vectors y are uncorrelated**, which is what we want from the transform-domain coefficients.

The vector $\mathbf{b}$ can be recovered as

$$\mathbf{b} = \Gamma^{-1}\mathbf{y}$$

and hence the original block is reconstructed using $\mathbf{x} = \Gamma^{-1}\mathbf{y} + \mu$.

Suppose that instead of using all the n^2 eigenvectors of $\mathbf{R}_b$, we use only k eigenvectors corresponding to the k largest eigenvalues and form a transformation matrix Γ_k of order $k \times n^2$. The resulting transformed vector $\hat{\mathbf{y}}$ therefore becomes k-dimensional and the reconstruction $\hat{\mathbf{x}} = \Gamma_k^T\hat{\mathbf{y}} + \mu$ will not be exact.

4.1 Optimality of the K-L transform

If we project the mean-removed vector $\mathbf{b}$ into any of the eigenvectors $\mathbf{e}_j$, the projection is defined by the inner product of the vectors $\mathbf{b}$ and $\mathbf{e}_j$, and is given by

$$A = \mathbf{b}^T \mathbf{e}_j = \mathbf{e}_j^T \mathbf{b}$$

The variance σ^2 of the projection is therefore given by

$$\sigma^2 = \mathbb{E}[A^2] = \mathbb{E}[\mathbf{e}_j^T \mathbf{b}\mathbf{b}^T \mathbf{e}_j] = \mathbf{e}_j^T \mathbb{E}[\mathbf{b}\mathbf{b}^T]\mathbf{e}_j = \mathbf{e}_j^T \mathbf{R}_b \mathbf{e}_j = \lambda_j$$

By projecting the vector $\mathbf{b}$ into all the n^2 eigenvectors, we obtain the total variance as $\sigma_{n^2}^2 = \sum_{j=1}^{n^2} \lambda_j$. By considering only the first k eigenvectors, the variance of the approximating signal in the projected space is given by $\sigma_k^2 = \sum_{j=1}^{k} \lambda_j$ (which is where the ordering comes into play, as this is the choice of the k largest eigenvalues).

Thus, the mean-square error in the projected space by considering only the first k components can be obtained as

$$e_{ms} = \sigma_{n^2}^2 - \sigma_k^2 = \sum_{j=k+1}^{n^2} \lambda_j$$

Since, the transformation is energy-preserving, the same mean-square error exists between the original vector $\mathbf{x}$ and its approximation $\mathbf{x}$. Since the λ_js decrease monotonically, we can minimize this error by choosing the largest k eigenvalues.

Thus, K-L transform is optimal in the sense that it minimizes the mean-square error between the original input vectors $\mathbf{x}$ and their approximations $\hat{\mathbf{x}}$.

5 The Discrete Cosine Transform

DCT is an orthogonal transformation that is very widely used in image compression and is widely accepted in the multimedia standards. The type-2 DCT transforms a block of image of size $N \times N$ having pixel intensities $s(n_1, n_2)$ into a transform array of coefficients $S(k_1, k_2)$:

$$S(k_1, k_2) = \frac{2}{N} C(k_1) C(k_2) \sum_{n_1=0}^{N-1} \sum_{n_2=0}^{N-1} s(n_1, n_2) \cos\left(\frac{\pi(2n_1+1)k_1}{2N}\right) \cos\left(\frac{\pi(2n_2+1)k_2}{2N}\right)$$

where $k_1, k_2, n_1, n_2 \in \{0, 1, \ldots, N-1\}$, and $C(k) = \frac{1}{\sqrt{2}}$ if $k = 0$ and 1 otherwise. The transform-domain indices k_1 and k_2 indicate the spatial frequencies in the directions of n_1 and n_2 respectively. $k_1 = k_2 = 0$ corresponds to the average or the DC component and all the remaining ones are the AC components which correspond to higher spatial frequencies as k_1 and k_2 increase.

Computation. Directly applying the above formula gives us $\mathcal{O}(N^4)$ computations. But faster algorithms exist which allow the computations to be done in just $\mathcal{O}(N^2 \log N)$ time.

6 Fast Computation of the 2D DCT-II: Martin Vetterli

The DCT of $N \times N$ is mapped into a real DFT of $N \times N$ followed by post-multiplications. These post-multiplications are shown to be rotations, thus reducing the required number of operations. Here we will remain loyal to the notation used in the manuscript.

Assume a real signal of dimension $N \times N$. The 2D DCT is defined as

$$\mathrm{DCT}(k_1, k_2) = \sum_{n_1=0}^{N-1} \sum_{n_2=0}^{N-1} x(n_1, n_2) \cos\left(\frac{2\pi(2n_1+1)k_1}{4N}\right) \cos\left(\frac{2\pi(2n_2+1)k_2}{4N}\right)$$

Define

$$y(n_1, n_2) = \begin{cases} x(2n_1, 2n_2) & n_1 = 0, \ldots, N/2-1; \quad n_2 = 0, \ldots, N/2-1 \\ x(2N-2n_1-1, 2n_2) & n_1 = N/2, \ldots, N-1; \quad n_2 = 0, \ldots, N/2-1 \\ x(2n_1, 2N-2n_2-1) & n_1 = 0, \ldots, N/2-1; \quad n_2 = N/2, \ldots, N-1 \\ x(2N-2n_1-1, 2N-2n_2-1) & n_1 = N/2, \ldots, N-1; \quad n_2 = N/2, \ldots, N-1 \end{cases}$$

Then the DCT becomes

$$\mathrm{DCT}(k_1,k_2)=\sum_{n_1=0}^{N-1}\sum_{n_2=0}^{N-1}y(n_1,n_2)\cos\left(\frac{2\pi(4n_1+1)k_1}{4N}\right)\cos\left(\frac{2\pi(4n_2+1)k_2}{4N}\right)$$

Now we rewrite the same in terms of

$$\text{cos-DFT}(k_1,k_2,N)=\sum_{n_1=0}^{N-1}\sum_{n_2=0}^{N-1}y(n_1,n_2)\cos\left(\frac{2\pi(n_1k_1+n_2k_2)}{N}\right)$$

and

$$\text{sin-DFT}(k_1,k_2,N)=\sum_{n_1=0}^{N-1}\sum_{n_2=0}^{N-1}y(n_1,n_2)\sin\left(\frac{2\pi(n_1k_1+n_2k_2)}{N}\right)$$

which correspond to the real and imaginary parts of a 2-D DFT:

$$\begin{aligned}\mathrm{DCT}(k_1,k_2,N)=\frac{1}{2}\Bigg[&\cos\left(\frac{2\pi(k_1+k_2)}{4N}\right)\text{cos-DFT}(k_1,k_2,N)\\&-\sin\left(\frac{2\pi(k_1+k_2)}{4N}\right)\text{sin-DFT}(k_1,k_2,N)\\&+\cos\left(\frac{2\pi(k_1-k_2)}{4N}\right)\text{cos-DFT}(k_1,N-k_2,N)\\&-\sin\left(\frac{2\pi(k_1-k_2)}{4N}\right)\text{sin-DFT}(k_1,N-k_2,N)\Bigg]\end{aligned}$$

Now, we note that $\mathrm{DCT}(k_1,k_2,N),\mathrm{DCT}(N-k_1,k_2,N),\mathrm{DCT}(k_1,N-k_2,N)$ and $\mathrm{DCT}(N-k_1,N-k_2,N)$ can be derived from the corresponding cos- and sin-DFTs by 2 plane rotations and some additions as follows:

$$\begin{bmatrix}\mathrm{DCT}(k_1,k_2,N)\\ \mathrm{DCT}(N-k_1,k_2,N)\\ \mathrm{DCT}(k_1,N-k_2,N)\\ \mathrm{DCT}(N-k_1,N-k_2,N)\end{bmatrix}=\begin{bmatrix}1&0&1&0\\0&1&0&1\\0&1&0&-1\\-1&0&1&0\end{bmatrix}\begin{bmatrix}\mathbf{R}(k_1,k_2)&\mathbf{O}_2\\ \mathbf{O}_2&\mathbf{R}(k_1,-k_2)\end{bmatrix}\begin{bmatrix}\text{cos-DFT}(k_1,k_2,N)\\ \text{sin-DFT}(k_1,k_2,N)\\ \text{cos-DFT}(k_1,N-k_2,N)\\ \text{sin-DFT}(k_1,N-k_2,N)\end{bmatrix}$$

where

$$\mathbf{O}_2=\begin{bmatrix}0&0\\0&0\end{bmatrix}$$

and

$$\mathbf{R}(k_1,k_2)=\begin{bmatrix}\cos\left(\frac{2\pi(k_1+k_2)}{4N}\right)&-\sin\left(\frac{2\pi(k_1+k_2)}{4N}\right)\\ \sin\left(\frac{2\pi(k_1+k_2)}{4N}\right)&\cos\left(\frac{2\pi(k_1+k_2)}{4N}\right)\end{bmatrix}$$

The real DFT will be computed using polynomial transforms. Below, the main steps are simply recalled. We want to evaluate:

$$X(k_1,k_2)=\sum_{n_1=0}^{N-1}\sum_{n_2=0}^{N-1}x(n_1,n_2)W^{n_1k_1}W^{n_2k_2}\qquad W=e^{j2\pi/N}$$

where $N=2^m$. Since $x(n_1,n_2)$ is real, $X(k_1,k_2)$ is equal to $X(N-k_1,N-k_2)^*$, $*$ denoting complex conjugates.

For k_2 odd we can write:

$$X(k_1,k_2)\equiv X_{k_1}^1(z)\pmod{(z-W^{k_2})}\qquad k_2\text{ odd}$$

where

$$X^1_{k_1}(z) \equiv \sum_{n_1=0}^{N-1} X^1_{n_1}(z) W^{n_1 k_1} \pmod{(z^{N/2}+1)}$$

and

$$X^1_{n_1}(z) = \sum_{n_2=0}^{N/2-1} \left[x(n_1, n_2) - x(n_1, n_2 + N/2)\right] z^{n_2}$$

Since $(z - W^{k_2})$, k_2 being odd, is a factor of $(z^{N/2}+1)$, which is in turn a factor of $(z^N - 1)$. Now, k_1 is replaced by $k_1 k_2$ (which is a valid permutation since k_2 and N are relatively prime). Then, we can replace W^{k_2} by z since they are equivalent modulo 10. This leads to:

$$X^1_{k_1,k_2}(z) \equiv \sum_{n_1=0}^{N-1} X^1_{n_1}(z) z^{n_1 k_1} \pmod{(z^{N/2}+1)}$$

This is a polynomial transform of length N and a root z defined modulo $(z^{N/2}+1)$ which can be computed with an FFT radix-2 type algorithm and uses $N^2 \log_2 N$ real additions for $X^1_{n_1}$ with real coefficients. Since $X^1_{k_1 k_2}(z)$ is of degree $N/2-1$ and k_2 odd equal to $(2u+1)$,:

$$X(k_1(2u+1), (2u+1)) = \sum_{l=0}^{N/2-1} y(k_1, l) W^{2ul}$$

where $y(k_1, l)$ is the l-th coefficient of the polynomial $X^1_{k_1 k_2}(z)$.
When both k_1 and k_2 are even, we get a real DFT of size $N/2 \times N/2$ on the real signal given by:

$$x(n_1, n_2) + x(n_1, n_2 + N/2) + x(n_1 + N/2, n_2) + x(n_1 + N/2, n_2 + N/2)$$

When k_1 is odd and k_2 even, we get

$$X(k_1, 2u\, k_1) \equiv X^1_{2u\, k_1}(z) \pmod{(z - W^{k_1})} \qquad k_1 \text{ odd}$$

where

$$X^1_{2u\, k_1}(z) \equiv \sum_{n_2=0}^{N-1} X^1_{n_2}(z) z^{2un_2} \pmod{(z^{N/2}+1)} \qquad u = 0, \ldots, N/2-1$$

and

$$X^1_{n_2}(z) = \sum_{n_1=0}^{N/2-1} \Big[x(n_1, n_2) + x(n_1, n_2 + N/2)$$

$$-x(n_1 + N/2, n_2) - x(n_1 + N/2, n_2 + N/2)\Big] z^{n_1}$$

All together, the real DFT of N by N has been mapped into $3N/2$ odd DFTs of length $N/2$ on real data and a real DFT of $N/2$ by $N/2$, at the cost of $3N^2/4 \log_2 N + 5N^2/4$ real additions.

Computation of the real odd DFTs.
The length-N odd DFTs required for the evaluation of 2-D DFTs have the following form:

$$X_k = \sum_{n=0}^{N-1} x(n) W_{2N}^{(2k+1)n} \qquad W_{2N} = e^{j2\pi/2N}$$

We introduce the following shorthands:

$$\text{ocos-DFT}(k, n) = \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi(2k+1)n}{2N}\right) \qquad k = 0, N-1$$

$$\text{osin-DFT}(k,n) = \sum_{n=0}^{N-1} x(n) \sin\left(\frac{2\pi(2k+1)n}{2N}\right) \qquad k = 0, N-1$$

$$\text{o-DCT}(k,n) = \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi(2k+1)(2n+1)}{4N}\right) \qquad k = 0, N-1$$

and thus

$$X_k = \text{ocos-DFT}(k,N) - j\,\text{osin-DFT}(k,N)$$

The real part is reduced to:

$$\text{ocos-DFT}(k,n) = \sum_{n=0}^{N/2-1} x(2n) \cos\left(\frac{2\pi(2k+1)n}{N}\right) + \sum_{n=0}^{N/4-1} [x(2n+1) - x(2n-1)] \cos\left(\frac{2\pi(2k+1)(2n+1)}{2N}\right)$$

where we use the fact that $\text{o-DCT}(2N-k-1, N) = -\text{o-DCT}(k,N)$. Thus, with $N/4$ input and $N/2$ output additions, the odd cos-DFT of N has been reduced to one of $N/2$ and an odd DCT of $N/4$. The imaginary part becomes:

$$\text{osin-DFT}(k,n) = \sum_{n=0}^{N/2-1} x(2n) \sin\left(\frac{2\pi(2k+1)n}{N}\right)$$

$$+ \sum_{n=0}^{N/4-1} (x(2n+1) + x(N-2n-1)) \sin\left(\frac{2\pi(2k+1)(2n+1)}{2N}\right)$$

$$\implies \text{osin-DFT}(k,n) = \sum_{n=0}^{N/2-1} x(2n) \sin\left(\frac{2\pi(2k+1)n}{N}\right)$$

$$+ \sum_{n=0}^{N/4-1} (-1)^n (x(2n+1) + x(N-2n-1)) \cos\left(\frac{2\pi(N/2-(2k+1))(2n+1)}{2N}\right)$$

or an odd sin-DFT of $N/2$ and an odd DCT of $N/4$ at the cost of $3N/4$ additions. Turning to the computation of the odd DCT, we use:

$$y(n) = x(2n) \qquad y(N-n-1) = -x(2n+1)$$

Thus:

$$\text{o-DCT}(k,n) = \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi(4k+1)(4n+1)}{8N}\right), \qquad k = 0 \ldots N-1$$

Fast 2-D DCT Algorithm (Vetterli, 1985)

Input: A real signal $x(n_1, n_2)$ of dimension $N \times N$, where N is a power of 2.

Step 1 — Reindex the input into $y(n_1, n_2)$

Rearrange x into a new array y of the same size using:

$$y(n_1,n_2) = x(2n_1, 2n_2), \quad n_1, n_2 = 0 \ldots N/2-1$$

$$y(n_1,n_2) = x(2N-2n_1-1, 2n_2), \quad n_1 = N/2 \ldots N-1,\ n_2 = 0 \ldots N/2-1$$

$$y(n_1,n_2) = x(2n_1, 2N-2n_2-1), \quad n_1 = 0 \ldots N/2-1,\ n_2 = N/2 \ldots N-1$$

$$y(n_1,n_2) = x(2N-2n_1-1, 2N-2n_2-1), \quad n_1, n_2 = N/2 \ldots N-1$$

Step 2 — Compute the 2-D DFT of y via polynomial transforms

We want to evaluate the 2-D DFT:

$$X(k_1, k_2) = \sum_{n_1=0}^{N-1} \sum_{n_2=0}^{N-1} x(n_1, n_2)\, W^{n_1 k_1} W^{n_2 k_2}, \qquad W = e^{-j2\pi/N}$$

The computation is split into three cases depending on the parity of k_1 and k_2.

Case A — both k_1 and k_2 even:

Equation (9) reduces directly to a real DFT of size $N/2 \times N/2$ on the real signal:

$$\tilde{x}(n_1, n_2) = x(n_1, n_2) + x(n_1, n_2 + N/2) + x(n_1 + N/2, n_2) + x(n_1 + N/2, n_2 + N/2)$$

Case B — k_2 odd (any k_1):

Since $(z - W^{k_2})$ with k_2 odd is a factor of $(z^{N/2} + 1)$, equation (9) can be written as:

$$X(k_1, k_2) \triangleq X^1_{k_1}(z) \bmod (z - W^{k_2}), \qquad k_2 \text{ odd}$$

where the intermediate polynomial is:

$$X^1_{k_1}(z) \triangleq \sum_{n_1=0}^{N-1} X^1_{n_1}(z)\, W^{n_1 k_1} \bmod (z^{N/2} + 1)$$

and the inner polynomial is formed by combining conjugate rows:

$$X^1_{n_1}(z) = \sum_{n_2=0}^{N/2-1} \left[x(n_1, n_2) - x(n_1, n_2 + N/2)\right] z^{n_2}$$

The key step is to replace k_1 by $k_1 k_2$ (a valid permutation since k_2 and N are relatively prime) and then replace W^{k_2} by z, since they are equivalent under the modular reduction. This yields the polynomial transform:

$$X^1_{k_1 k_2}(z) \triangleq \sum_{n_1=0}^{N-1} X^1_{n_1}(z)\, z^{n_1 k_1} \bmod (z^{N/2} + 1)$$

This is a length-N polynomial transform with root z defined modulo $(z^{N/2} + 1)$, computable with an FFT radix-2 type algorithm. Since k_2 is odd, write $k_2 = 2u + 1$; then equation (10) becomes:

$$X\big(k_1(2u+1),\, (2u+1)\big) = \sum_{l=0}^{N/2-1} y(k_1, l)\, W^l\, W^{2ul}$$

where $y(k_1, l)$ is the l-th coefficient of the polynomial $X^1_{k_1 k_2}(z)$. This represents N odd DFTs of length $N/2$ to be computed as in Step 3.

Case C — k_1 odd, k_2 even:

Equation (9) can be rewritten similarly to Case B as:

$$X(k_1, 2uk_1) \triangleq X^1_{2uk_1}(z) \bmod (z - W^{k_1}), \qquad k_1 \text{ odd}$$

where now the length-$N/2$ polynomial transform is defined modulo $(z^{N/2} + 1)$:

$$X^1_{2uk_1}(z) \triangleq \sum_{n_2=0}^{N-1} X^1_{n_2}(z)\, z^{2un_2} \bmod (z^{N/2} + 1), \qquad u = 0 \ldots N/2 - 1$$

and the inner polynomial combines conjugate columns:

$$X^1_{n_2}(z) = \sum_{n_1=0}^{N/2-1} \big[x(n_1,n_2) + x(n_1,n_2+N/2) - x(n_1+N/2,n_2) - x(n_1+N/2,n_2+N/2)\big]\, z^{n_1}$$

This is a length-$N/2$ polynomial transform with root z^2 defined modulo $(z^{N/2}+1)$, computable similarly to Case B. The result is then evaluated as $N/2$ odd DFTs of length $N/2$ as in Step 3.

Step 3 — Compute the real odd DFTs

Each length-N odd DFT has the form:

$$X_k = \sum_{n=0}^{N-1} x(n)\, W_{2N}^{(2k+1)n}, \qquad W_{2N} = e^{-j2\pi/2N}$$

Split into real and imaginary parts:

$$X_k = \text{ocos-DFT}(k,N) - j\,\text{osin-DFT}(k,N)$$

Step 3a — Compute ocos-DFT by decimation:

$$\text{ocos-DFT}(k,n) = \sum_{n=0}^{N/2-1} x(2n)\cos\left(\frac{2\pi(2k+1)n}{N}\right) + \sum_{n=0}^{N/4-1} \big(x(2n{+}1) - x(N{-}2n{-}1)\big)\cos\left(\frac{2\pi(2k+1)(2n+1)}{2N}\right)$$

This reduces to an odd cos-DFT of length $N/2$ plus an odd DCT of length $N/4$.

Step 3b — Compute osin-DFT using trigonometric identities:

$$\text{osin-DFT}(k,n) = \sum_{n=0}^{N/2-1} x(2n)\sin\left(\frac{2\pi(2k+1)n}{N}\right) + \sum_{n=0}^{N/4-1} (-1)^n\big(x(2n{+}1) + x(N{-}2n{-}1)\big)\cos\left(\frac{2\pi(N/2-(2k+1))(2n+1)}{2N}\right)$$

This reduces to an odd sin-DFT of length $N/2$ and an odd DCT of length $N/4$.

Step 3c — Compute the odd DCT via the mapping $y(n) = x(2n)$, $y(N-n-1) = -x(2n+1)$, giving:

$$\text{o-DCT}(k,n) = \sum_{n=0}^{N-1} x(n)\cos\left(\frac{2\pi(4k+1)(4n+1)}{8N}\right), \quad k = 0\ldots N-1$$

Step 4 — Post-multiply to recover the DCT

From the cos- and sin-DFTs, recover four DCT quadrants simultaneously using two plane rotations per (k_1,k_2) pair:

$$\begin{bmatrix} \text{DCT}(k_1,k_2) \\ \text{DCT}(N{-}k_1,k_2) \\ \text{DCT}(k_1,N{-}k_2) \\ \text{DCT}(N{-}k_1,N{-}k_2) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ -1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R(k_1,k_2) & O_2 \\ O_2 & R(k_1,-k_2) \end{bmatrix} \begin{bmatrix} \text{ocos-DFT}(k_1,k_2) \\ \text{osin-DFT}(k_1,k_2) \\ \text{ocos-DFT}(k_1,N{-}k_2) \\ \text{osin-DFT}(k_1,N{-}k_2) \end{bmatrix}$$

where $R(k_1,k_2)$ is the 2×2 rotation matrix:

$$R(k_1,k_2) = \begin{bmatrix} \cos\left(\dfrac{2\pi(k_1+k_2)}{4N}\right) & -\sin\left(\dfrac{2\pi(k_1+k_2)}{4N}\right) \\ \sin\left(\dfrac{2\pi(k_1+k_2)}{4N}\right) & \cos\left(\dfrac{2\pi(k_1+k_2)}{4N}\right) \end{bmatrix}$$